Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

KALAHSTHI RAMYA
192311149RA

CO3-AT3-Debugging and Optimization

← Back

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q5 Railway Reservation – Synchronization Debugging — 10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:
1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {

int seats = 1;

void bookSeat(String customer) {

if (seats > 0) {
```

```
System.out.println(customer
+ " is booking a seat");

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
System.out.println(customer
+ " - No seat available");
}
}
}

class Customer extends Thread {

Reservation reservation;
String customerName;

Customer(Reservation reservation, String
customerName) {
this.reservation = reservation;
this.customerName = customerName;
}

public void run() {
reservation.bookSeat(customerName);
}
}

class RailwayReservation {

public static void main(String[] args) {

Reservation reservation = new Reservation();
```

Customer c1 =
new Customer(reservation, "Customer 1");

Customer c2 =
new Customer(reservation, "Customer 2");

c1.start();
c2.start();
}
}

The corrected program must protect the shared seat resource using synchronization.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2
3 public class Main{
4     public static void main(String[] args) t
5         Reservation r = new Reservation();
6
7         Customer c1 = new Customer(r,"Custom
8         Customer c2 = new Customer(r,"Custom
9
10        c1.start();
11        c2.start();
12
13        c1.join();
14        c2.join();
15    }
16 }
17
18 class Reservation{
19    int seats = 1;
20
21    public synchronized void bookSeat(String
22        if(seats > 0){
23            System.out.println(customer + "
24            try{
```

```
                Thread.sleep(100);
            }catch(InterruptedException e){
                System.out.println("Thread i
            }
            seats--;
            System.out.println(customer+" bo
        }else{
            System.out.println(customer+" No
        }
    }
}

class Customer extends Thread{
    Reservation reservation;
    String customerName;

    Customer(Reservation reservation,String
        this.reservation = reservation;
        this.customerName = customerName;
    }
    public void run(){
        reservation.bookSeat(customerName);
    }
}
```

Test Input

stdin input (optional)

Output

```
Customer 1 is booking a seat
Customer 1 booking successful
Customer 2 No seat available
```

Pending manual review

Submitted